VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	1 to 15 (Combined Report)
Experiment Title	Luggage Complaint Portal: Complete DevOps Workflow (Combined Report)
Objectives (Skill Set / Knowledge Tested / Imparted)	To present the complete DevOps lifecycle of the Luggage Complaint Portal: planning, Git collaboration, continuous integration, continuous testing, containers and configuration management.
Program	**Summary of the 15 experiments**

No.	Experiment	Main tools
1	Problem Definition and Scope	Documents
2	Agile Planning and DevOps Workflow	Agile board, diagrams
3	Requirements, Architecture and Technology Setup	Spring Boot, Maven, Tomcat
4	Git and GitHub Repository Initialization	Git, GitHub
5	Feature Development with Branching	Git branches, pull request
6	MVP Completion and Git Collaboration	Git conflict, tags
7	Jenkins Installation and Continuous Integration Job	Jenkins, Maven
8	Pipeline as Code and Server Deployment	Jenkinsfile, Tomcat
9	Selenium Test Design and Local Execution	Selenium, Maven
10	Continuous Testing in Jenkins	Jenkins, Selenium
11	Docker Image and Container Lifecycle	Docker
12	Jenkins-Docker Continuous Deployment	Jenkins, Docker, registry
13	Configuration Management Script	Ansible
14	Automated Provisioning and Reliability Validation	Ansible
15	Final End-to-End Release, Documentation and Viva	All tools

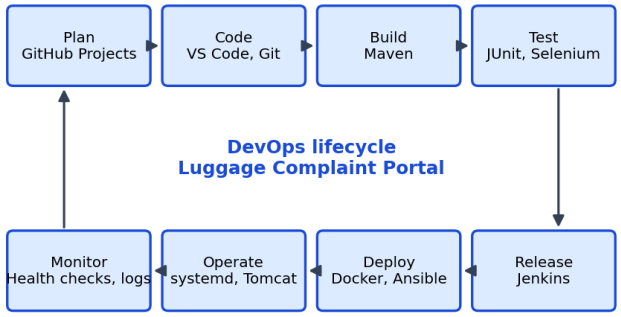


Figure: DevOps lifecycle followed in this project

Conclusion	The Luggage Complaint Portal was taken through the full DevOps workflow: requirements and planning, Git collaboration, a Jenkins pipeline with a Selenium quality gate, Docker packaging with a versioned registry, and Ansible-based provisioning.

VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	1
Experiment Title	Luggage Complaint Portal: Problem Definition and Scope
Objectives (Skill Set / Knowledge Tested / Imparted)	To study the need for the Luggage Complaint Portal, identify users, pain points, stakeholders and constraints, set measurable success criteria and freeze a small 15-task MVP scope.
Program	

1. Problem statement

Passengers whose luggage is delayed, lost or damaged currently report complaints through phone calls, paper forms or scattered emails. There is no single place to submit a complaint, attach proof, or track its status. This causes delays, lost information and frustration. The Luggage Complaint Portal is a web application where passengers submit and track complaints, and airline staff review and approve or reject them.

2. Target users and existing pain points

User	Pain point
Passenger	No easy way to file a complaint, attach proof or know the status
Reviewer (airline staff)	Complaints arrive in different formats and are hard to track
Management	No data on complaint volume or resolution time

3. Stakeholder list

Stakeholder	Role / interest
Passengers	Submit and track complaints
Reviewers (airline staff)	Review, approve or reject complaints
Airline management	Monitor service quality
Development team	Build and maintain the portal
DevOps / operations	Deploy and keep the system running
Faculty / examiner	Evaluate the project

	4. Objectives and measurable success criteria 1. A passenger can submit a complaint with validation and file upload in under 3 minutes. 2. A passenger can track a complaint using its reference number. 3. A reviewer can log in and approve or reject a complaint. 4. Every code push triggers an automated build and test (Jenkins). 5. The application runs the same way everywhere through Docker. 6. Unit and Selenium tests pass on every build. **5. Constraints** • Academic timeline and a single developer. • Free and open-source tools only (Git, Jenkins, Docker, Selenium, Ansible). • Runs on one Ubuntu VM with limited RAM. • No real airline integration, so demo data is used. • MVP only: no payments and no mobile app. **6. Approved MVP scope (frozen at 15 tasks)** In scope: complaint submission, validation, file upload, tracking, reviewer login, review list, approve/reject, automated tests, CI/CD, Docker, configuration management and documentation. Out of scope: email/SMS notifications, payments, analytics dashboard, mobile app and multi-language support.
Conclusion	The problem, users, stakeholders, objectives, constraints and a frozen 15-task MVP scope were defined, giving a clear baseline for the rest of the project.

VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	2
Experiment Title	Luggage Complaint Portal: Agile Planning and DevOps Workflow
Objectives (Skill Set / Knowledge Tested / Imparted)	To create user stories with acceptance criteria, a product backlog, a 15-task Kanban/Scrum plan, a Definition of Done and a DevOps lifecycle diagram.
Program	

1. User stories and acceptance criteria

ID	User story	Acceptance criteria
US1	As a passenger, I want to submit a complaint so that my issue is recorded	Required fields validated; a reference number is shown after submit
US2	As a passenger, I want to upload proof	Only allowed file types and sizes are accepted
US3	As a passenger, I want to track my complaint	Correct status shown; an invalid reference shows an error
US4	As a reviewer, I want to log in securely	Wrong credentials rejected; correct ones open the dashboard
US5	As a reviewer, I want to see all complaints	List shows reference, passenger, status and date
US6	As a reviewer, I want to approve or reject a complaint	Status changes and is visible to the passenger
US7	As a developer, I want automated tests	Unit and Selenium tests run on every build
US8	As a developer, I want automatic CI/CD	A push triggers the Jenkins build, test and package
US9	As operations, I want a Docker image	The app runs from a container with one command

2. Product backlog (priority order)

Priority	Item	Story points
1	Project skeleton and Git repository	2
2	Complaint submission and validation (US1)	5

3	File upload (US2)	3
4	Complaint tracking (US3)	3
5	Reviewer login (US4)	3
6	Review list and approve/reject (US5, US6)	5
7	Unit and Selenium tests (US7)	5
8	Jenkins pipeline (US8)	5
9	Dockerfile and container run (US9)	3
10	Ansible deployment	3
11	Documentation and viva preparation	2

3. 15-task Kanban board

To Do	In Progress	In Review	Done
Task 15: Final release, documentation and viva	Task 13: Configuration management Task 14: Automated provisioning		Tasks 1 to 12: problem definition, agile plan, architecture, Git/GitHub, branching, MVP and tag, Jenkins CI, pipeline as code, Selenium, continuous testing, Docker, Jenkins-Docker deployment

4. Sprint plan

Sprint	Goal	Tasks
Sprint 1	Plan and build the core features	Tasks 1 to 5
Sprint 2	Complete the MVP, test and automate	Tasks 6 to 10
Sprint 3	Containerise, provision and document	Tasks 11 to 15

5. Definition of Done

- Code is committed and pushed to GitHub (develop branch).
- Unit tests pass and the Jenkins build is green.
- Code is reviewed (pull request or self-review).
- Screenshots and a short report are saved.
- Documentation is updated.

6. DevOps lifecycle diagram

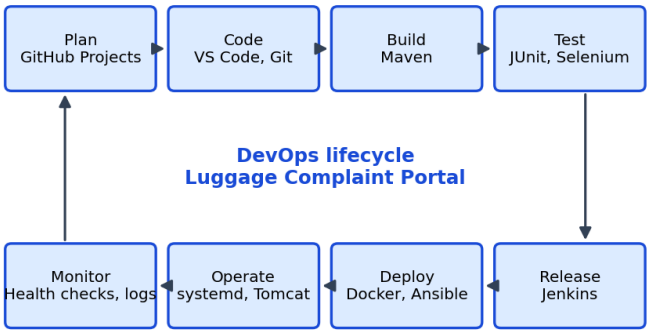


Figure: DevOps lifecycle from development to operations

Stage	Tool in this project
Plan	GitHub Projects / Kanban
Code	VS Code, Git, GitHub
Build	Maven
Test	JUnit, Selenium
Release	Jenkins pipeline
Deploy	Docker, Ansible
Operate / Monitor	systemd, Tomcat, health checks, Jenkins build history

Conclusion

User stories, a prioritised backlog, a 15-task board, a three-sprint plan, a Definition of Done and a DevOps lifecycle diagram were prepared, giving the project a clear agile structure.

VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	3
Experiment Title	Luggage Complaint Portal: Requirements, Architecture and Technology Setup
Objectives (Skill Set / Knowledge Tested / Imparted)	To design the minimum application architecture, select the technology stack, build tool and deployment target, and prepare a working local setup.
Program	**1. SRS summary**

ID	Functional requirement
FR1	A passenger submits a complaint with name, email, phone, flight number, PNR, bag tag, complaint type and description.
FR2	The system validates the data, shows error messages for invalid input and accepts a file upload.
FR3	The system generates a reference number in the format LCP-YYYY-NNNNNN.
FR4	A passenger tracks the status and history of a complaint using the reference number.
FR5	A reviewer logs in securely.
FR6	A reviewer sees the list of complaints.
FR7	A reviewer approves or rejects a complaint with a comment, and the status changes.

- Non-functional: simple responsive pages, reviewer pages protected by login, a health endpoint for monitoring, portability (WAR on Tomcat, Docker, systemd) and testability (unit and Selenium tests).

2. Use-case diagram

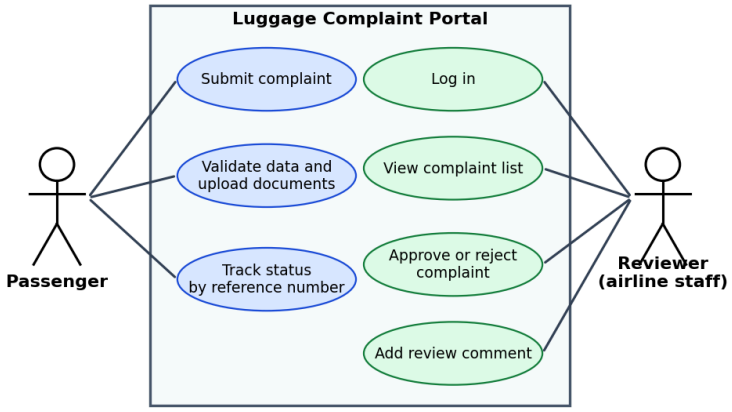


Figure: Use-case diagram (Passenger and Reviewer)

3. Architecture diagram

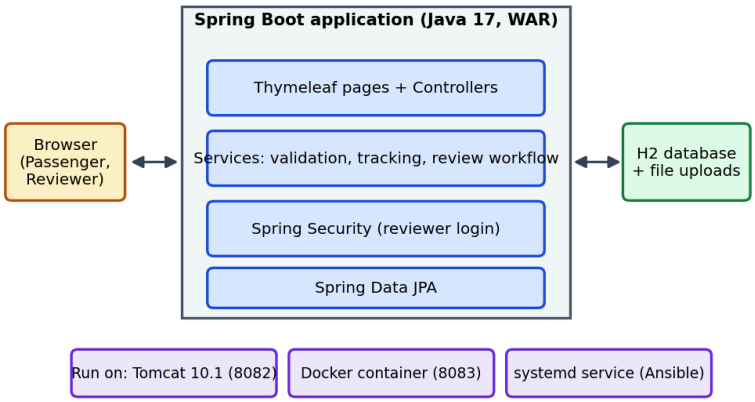


Figure: Minimum application architecture and deployment targets

4. Technology stack

Item	Choice
Language	Java 17
Framework	Spring Boot 3.3.5 with Thymeleaf, Spring Security and Spring Data JPA
Database	H2 (embedded)
Build tool	Maven (packaged as a WAR)
Deployment target	Tomcat 10.1 (also Docker and systemd)
Testing	JUnit 5 and Selenium WebDriver 4

5. Data model

Entity	Main fields

Complaint	reference number, passenger name, email, phone, flight number, PNR, bag tag, type, description, status, submitted date, attachment
Status history	when, from status, to status, changed by, note
Reviewer account	username and credentials used for the reviewer login

6. URL / API list

Method and path	Purpose
GET /	Home page with Submit, Track and Reviewer links
GET /complaints/new	Complaint form (submitted by POST from the same form)
GET /track	Track a complaint by reference number
GET /login	Reviewer login
Reviewer pages (after login)	Complaint list and approve/reject decision form
GET /health	Plain health check (returns OK)
GET /actuator/health, /actuator/info	Spring Boot health and build information

7. Working local setup

```
cd luggage-complaint-portal
mvn spring-boot:run
# open http://localhost:8081

Tomcat started on port 8081 (http) with context path '/'
Started LuggagePortalApplication in 5.996 seconds
```

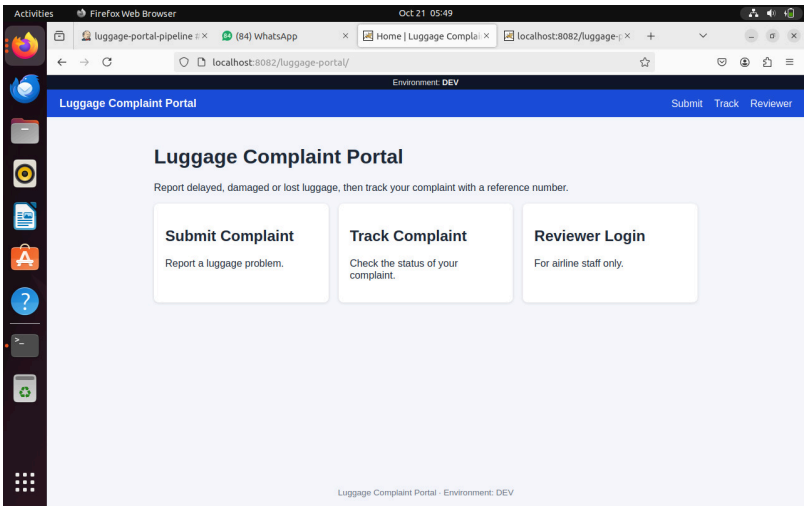


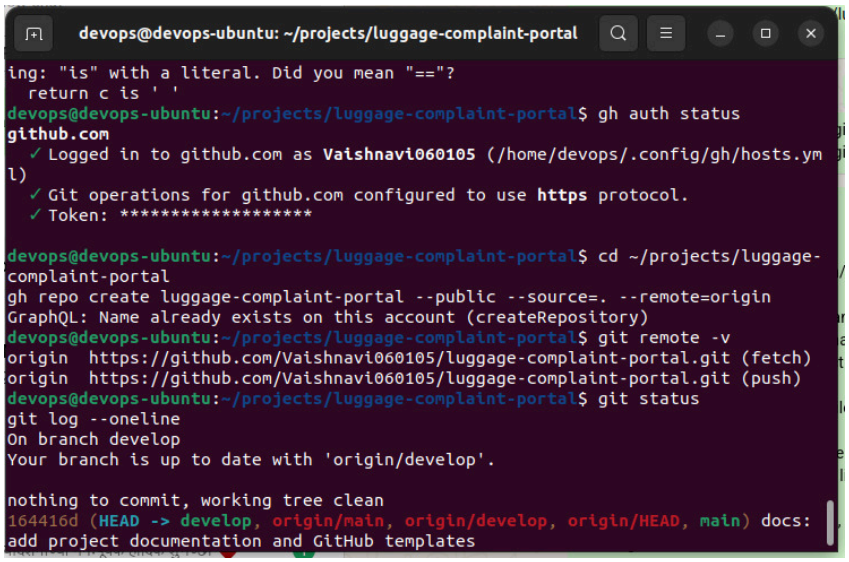
Figure: Application running (home page with environment banner)

Conclusion	The minimum architecture, technology stack, data model and URLs were defined, and the application was set up and run locally.

Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	4
Experiment Title	Luggage Complaint Portal: Git and GitHub Repository Initialization
Objectives (Skill Set / Knowledge Tested / Imparted)	To create a GitHub repository with README, .gitignore, folder structure, issue templates and branch rules, and to commit the initial application skeleton with meaningful messages.
Program	**1. GitHub repository** Repository URL: https://github.com/Vaishnavi060105/luggage-complaint-portal  *Figure: GitHub CLI login, remote configuration and commit log*

2. Folder structure on GitHub

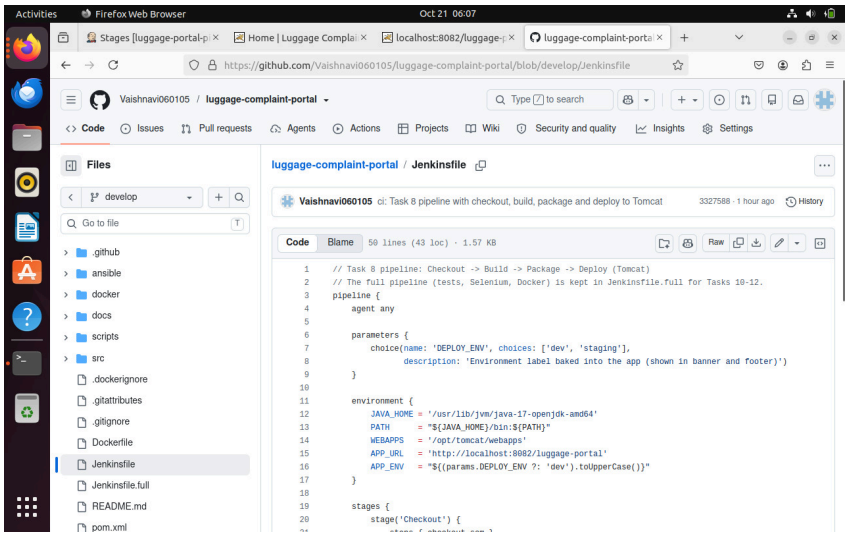


Figure: Repository on GitHub (develop branch) with .github, ansible, docker, docs, scripts and src folders

3. Initial commits

```
164416d docs: add project documentation and GitHub templates
a4cfd35 ci: add Jenkinsfile, Dockerfile, Ansible playbook and helper
scripts
adc0cd8 test: add unit tests and Selenium journeys
9400a92 feat: reviewer login, review list and approve/reject workflow
8f01209 feat: complaint submission, validation, upload and tracking
1479a10 chore: initial Spring Boot skeleton
```

4. Branch policy

- main: stable, released code (tagged).
- develop: integration branch for finished features.
- feature/<name>: one branch per feature, merged into develop through a pull request.
- Commit messages use a prefix: feat, fix, docs, test, ci, chore.

[Screenshot to be added: README.md and the issue templates (.github folder) on GitHub]

Conclusion	The GitHub repository was created and the application skeleton was committed with meaningful messages, a clear folder structure, templates and a branch policy.

VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	5
Experiment Title	Luggage Complaint Portal: Feature Development with Branching
Objectives (Skill Set / Knowledge Tested / Imparted)	To implement a feature on a feature branch using add, commit, push, pull and log, raise a pull request, review the changes and merge into the development branch.

Program	**1. Create the feature branch**
	`git checkout develop`
	`git pull origin develop`
	`git checkout -b feature/health-endpoint`
	2. Implement the feature (/health endpoint)
	`@RestController`
	`public class HealthController {`
	`  @GetMapping("/health")`
	`  public String health() {`
	`  return "OK";`
	`  }`
	`}`
	`$ curl -i http://localhost:8081/health`
	`HTTP/1.1 200`
	`Content-Type: text/plain;charset=UTF-8`
	`Content-Length: 2`
	`OK`
	3. Add, commit and push
	`git add .`
	`git commit -m "feat: add /health endpoint for status checks"`
	`git push -u origin feature/health-endpoint`

4. Pull request

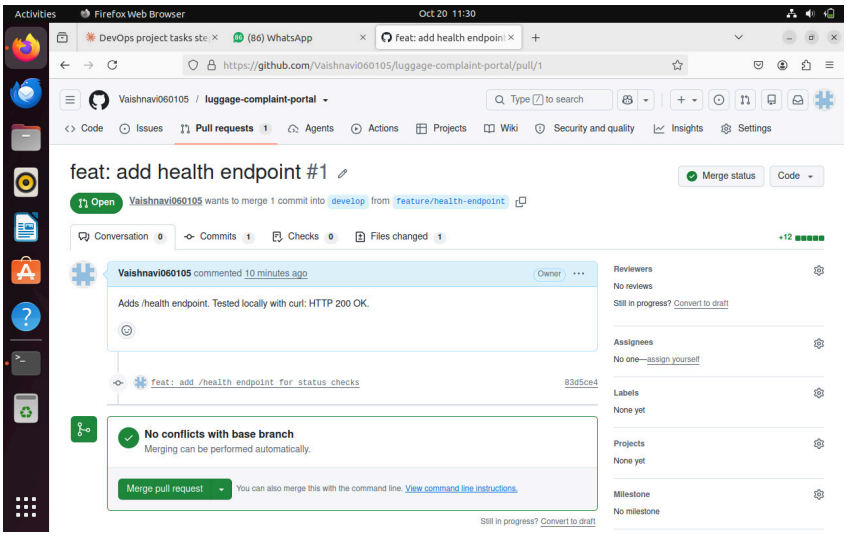


Figure: Pull request #1 from feature/health-endpoint into develop

5. Review comment

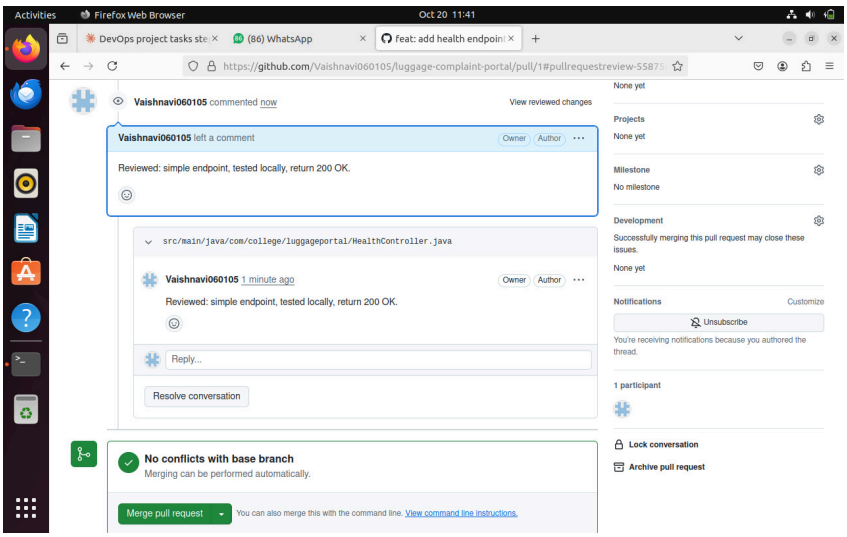
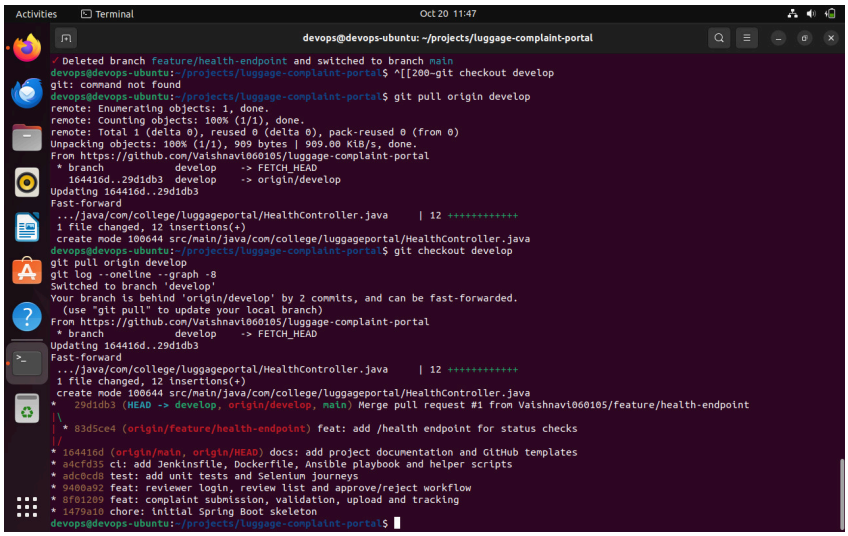


Figure: Review comment on HealthController.java

	6. Merge and evidence  *Figure: Merge into develop and git log --graph showing the merge commit 29d1db3*
Conclusion	A feature was developed on its own branch, raised as pull request #1, reviewed with a comment and merged into develop, showing the complete branching workflow.

Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	6
Experiment Title	Luggage Complaint Portal: MVP Completion and Git Collaboration
Objectives (Skill Set / Knowledge Tested / Imparted)	To complete the MVP functions, and to demonstrate a second feature branch, merge-conflict creation and resolution, tagging and a release-ready source baseline.
Program	**1. Two feature branches changing the same line**

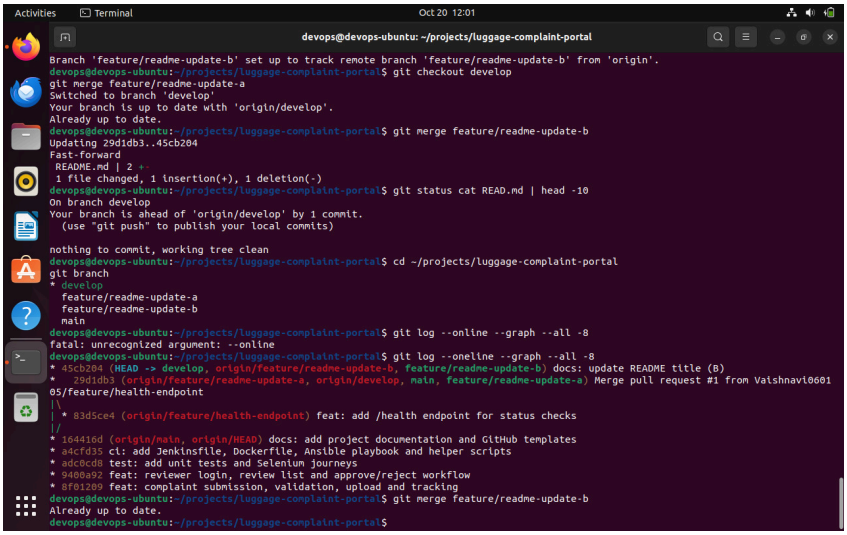


Figure: Branches feature/readme-update-a and feature/readme-update-b

2. Merge conflict

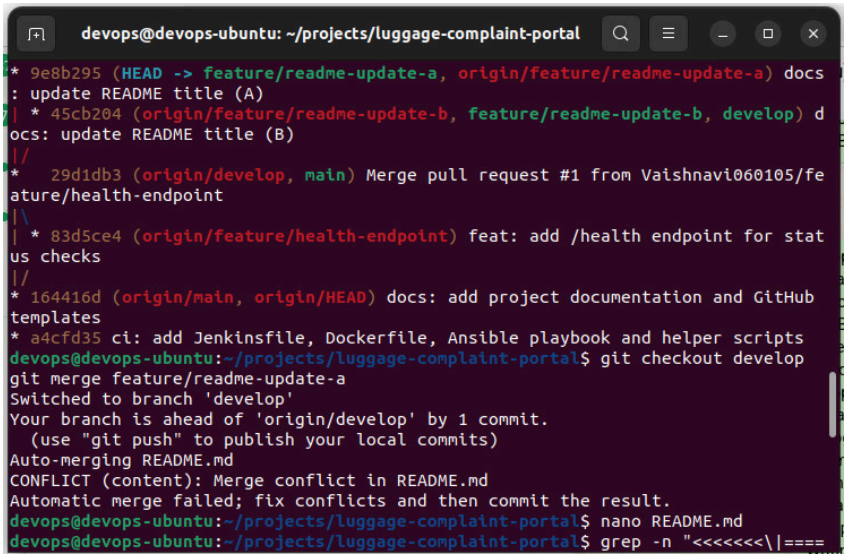


Figure: CONFLICT (content): Merge conflict in README.md

3. Conflict resolved

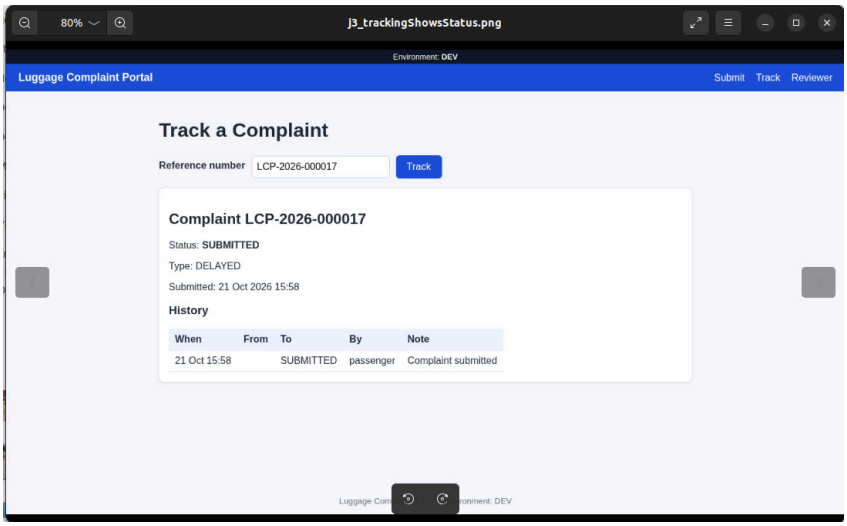
The conflict markers were removed in README.md, one title was kept, and the merge was committed and pushed.

```
*   53b018f (HEAD -> develop, origin/develop) merge: resolve README
conflict between A and B
|\
| * 9e8b295 (feature/readme-update-a) docs: update README title (A)
* | 45cb204 (feature/readme-update-b) docs: update README title (B)
|/
*   29d1db3 (main) Merge pull request #1 from
Vaishnavi060105/feature/health-endpoint
```

4. Tagged release baseline

```
git tag -a v1.0.0 -m "MVP release v1.0.0"
git push origin v1.0.0

$ git tag
v1.0.0
53b018f (HEAD -> develop, tag: v1.0.0, origin/main, origin/develop,
origin/HEAD, main) merge: resolve README conflict between A and B
```

	5. Functional MVP  *Figure: Track a complaint: reference LCP-2026-000017 with status SUBMITTED and history* ***[Screenshot to be added: Reviewer approving a complaint, the v1.0.0 tag on GitHub and the updated backlog]***
Conclusion	A second pair of feature branches produced a merge conflict that was resolved, the source baseline was tagged v1.0.0, and the MVP functions were demonstrated.

Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	7
Experiment Title	Luggage Complaint Portal: Jenkins Installation and Continuous Integration Job
Objectives (Skill Set / Knowledge Tested / Imparted)	To install and configure Jenkins, connect the GitHub repository, create a Maven build job triggered by commit polling, and archive the build artefact.
Program	

1. Jenkins installed

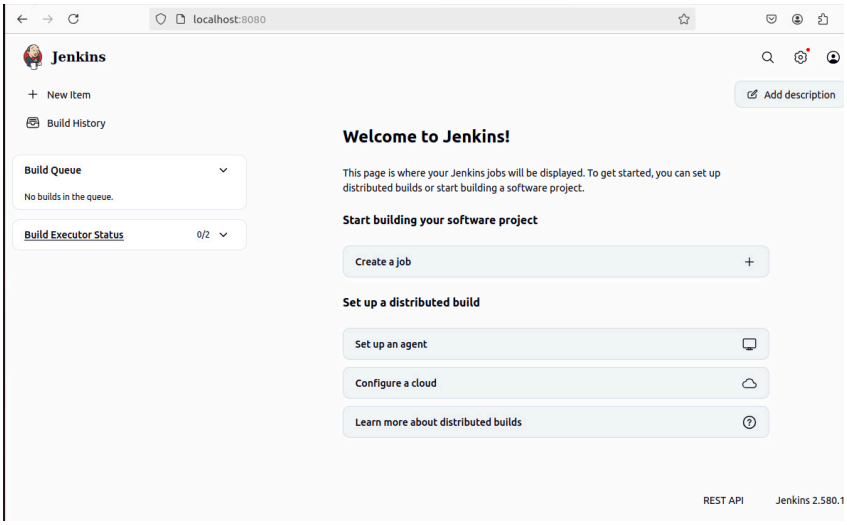


Figure: Jenkins dashboard after installation (Jenkins 2.580.1)

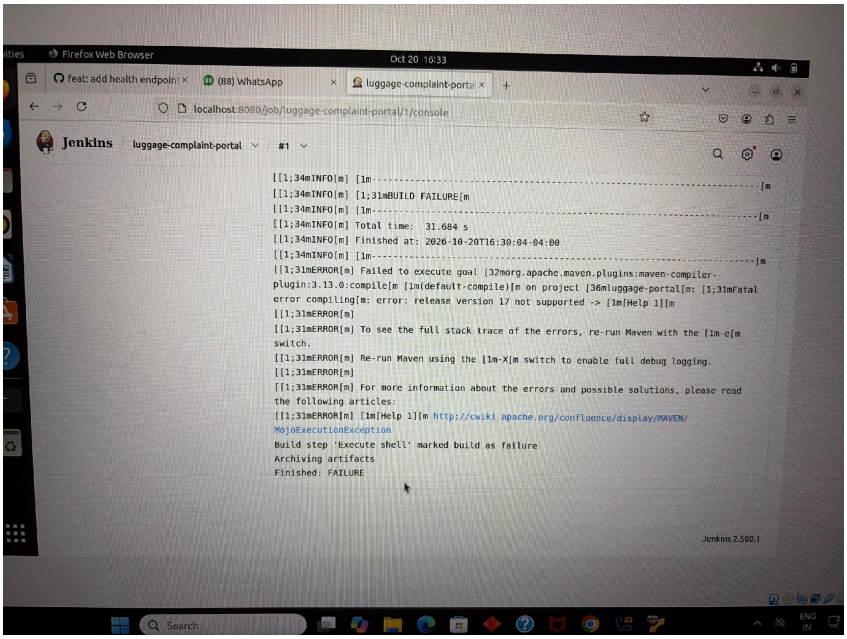
2. Job configuration

- Job type: Freestyle project (luggage-complaint-portal).
- Source: https://github.com/Vaishnavi060105/luggage-complaint-portal.git, branch */develop.
- Trigger: Poll SCM with schedule H/2 * * * *.
- Build step: Execute shell using Java 17, then mvn clean package -DskipTests.
- Post-build: Archive the artifacts target/*.war.

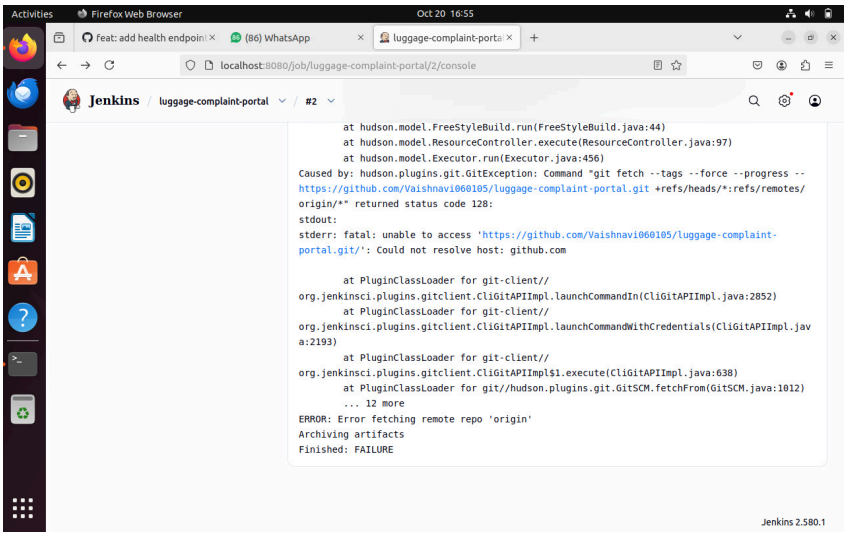
```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
```

```
mvn clean package -DskipTests
```

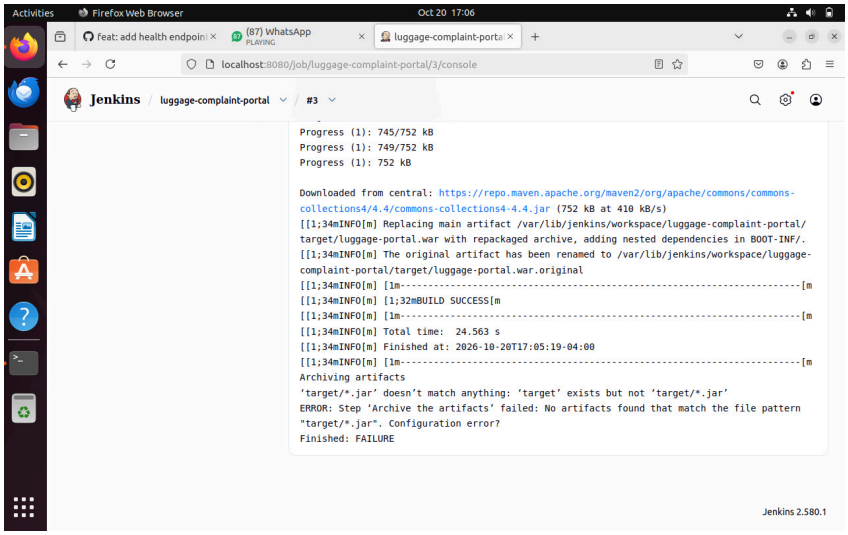
3. Failed builds and their fixes (troubleshooting)



Build #1: release version 17 not supported (default Java had no compiler); fixed by setting JAVA_HOME to Java 17



Build #2: Could not resolve host github.com (VM DNS); fixed by setting a public DNS



Build #3: BUILD SUCCESS but target/.jar not found; the project builds a WAR, so the pattern was changed to target/*.war*

4. Successful build and archived artefact

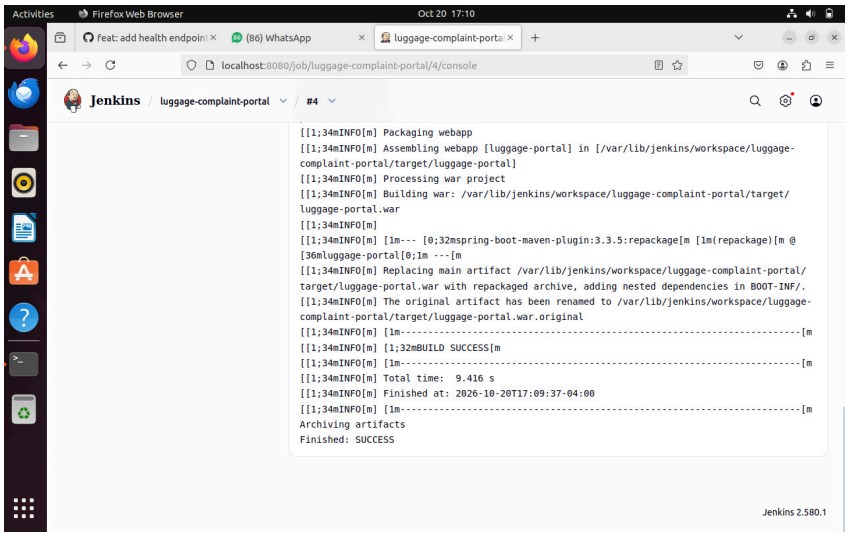


Figure: Console output of build #4 ending in BUILD SUCCESS

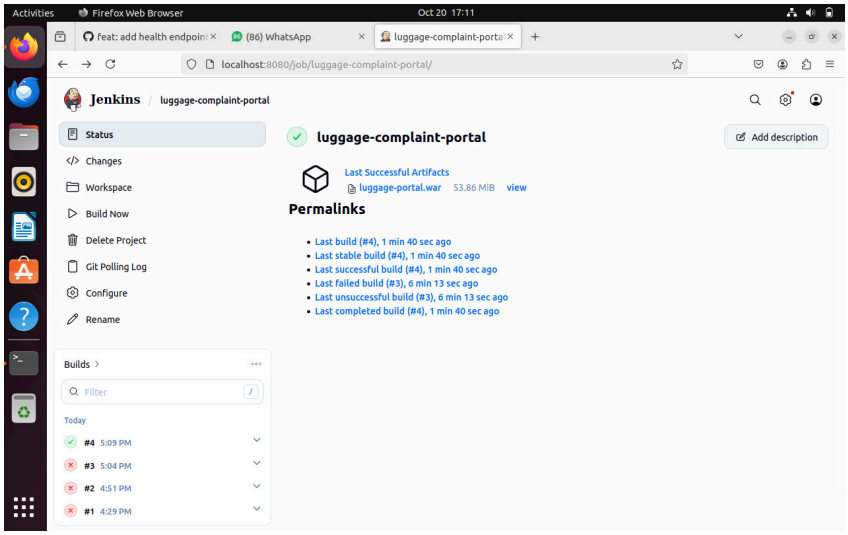


Figure: Job page with builds #1 to #4 and the archived luggage-portal.war

5. Trigger evidence

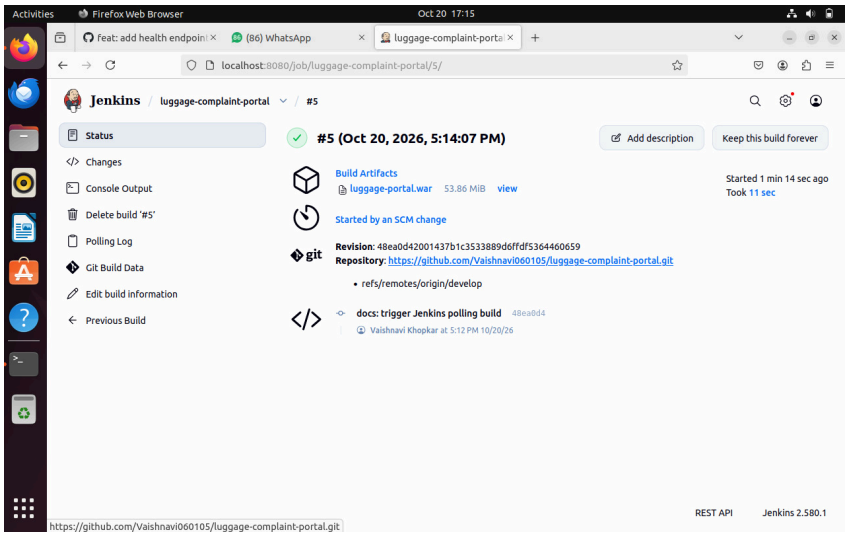


Figure: Build #5 started by an SCM change after a commit to develop

Conclusion	Jenkins was installed and a Maven CI job was connected to GitHub, started automatically by commits through SCM polling, and archived the WAR artefact. The failed builds were diagnosed and fixed.

VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	8
Experiment Title	Luggage Complaint Portal: Pipeline as Code and Server Deployment
Objectives (Skill Set / Knowledge Tested / Imparted)	To create a Jenkinsfile with checkout, build, package and deploy stages, deploy the build to Tomcat and parameterise an environment setting.
Program	

1. Tomcat installed as the deployment target

Apache Tomcat 10.1 was installed in /opt/tomcat, set to port 8082 (Jenkins uses 8080) and run as a systemd service. The jenkins user was added to the tomcat group so that it can copy the WAR into webapps.

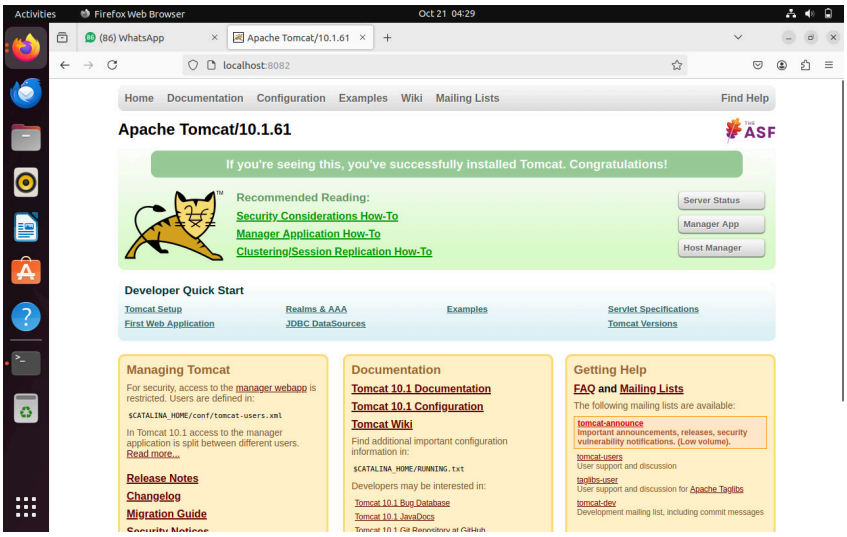


Figure: Tomcat 10.1 welcome page on port 8082

2. Jenkinsfile (pipeline as code)

```
pipeline {
    agent any
    parameters {
        choice(name: 'DEPLOY_ENV', choices: ['dev', 'staging'],
               description: 'Environment label baked into the app')
    }
    environment {
        JAVA_HOME = '/usr/lib/jvm/java-17-openjdk-amd64'
        WEBAPPS   = '/opt/tomcat/webapps'
        APP_URL   = 'http://localhost:8082/luggage-portal'
```

```
        APP_ENV    = "${(params.DEPLOY_ENV ?: 'dev').toUpperCase()}"
    }
    stages {
        stage('Checkout') { steps { checkout scm } }
        stage('Build')    { steps { sh 'mvn -B clean compile' } }
        stage('Package')  { steps { sh 'mvn -B package -DskipTests
-Dapp.env=$APP_ENV -Dapp.build=$BUILD_NUMBER' } }
        stage('Deploy')   { steps { sh 'cp target/luggage-portal.war
$WEBAPPS/luggage-portal.war' } }
    }
}
```

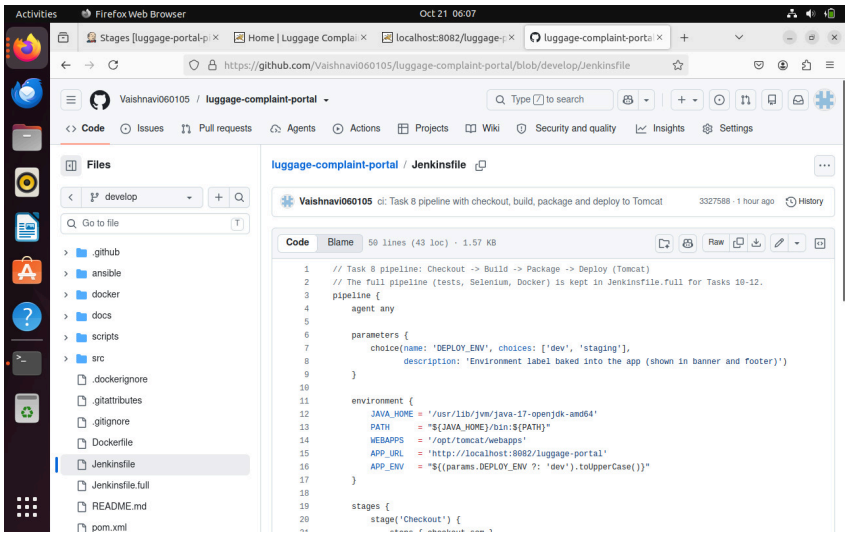


Figure: Jenkinsfile on GitHub (develop branch)

3. Pipeline run

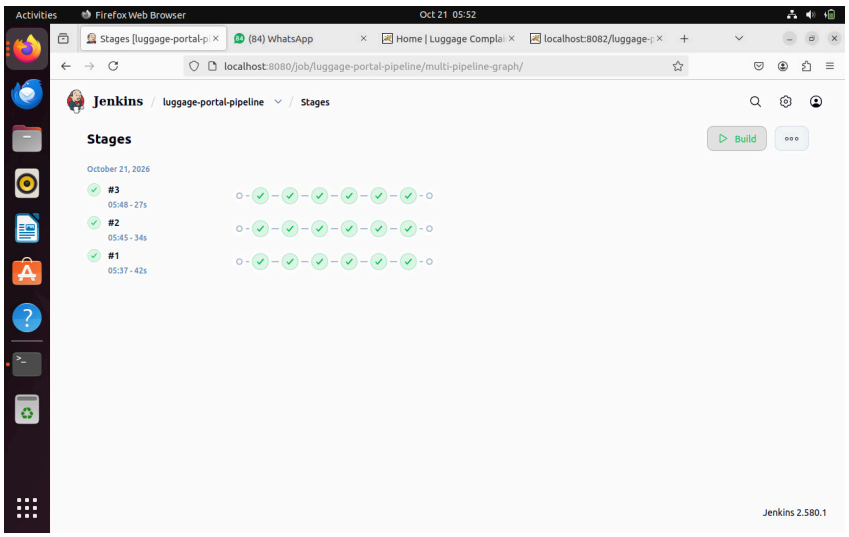


Figure: Stages view of builds #1 to #3, all stages green

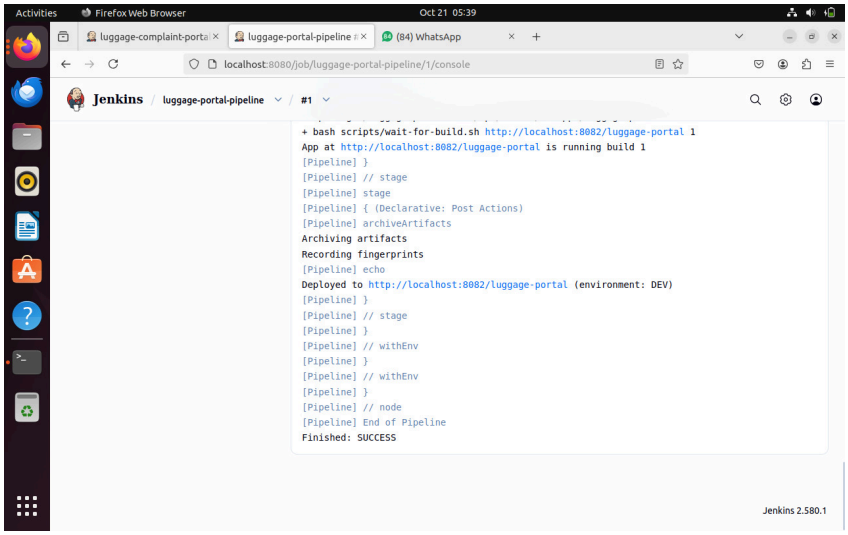


Figure: Console output: deployed to http://localhost:8082/luggage-portal (environment: DEV)

4. Parameterised environment setting

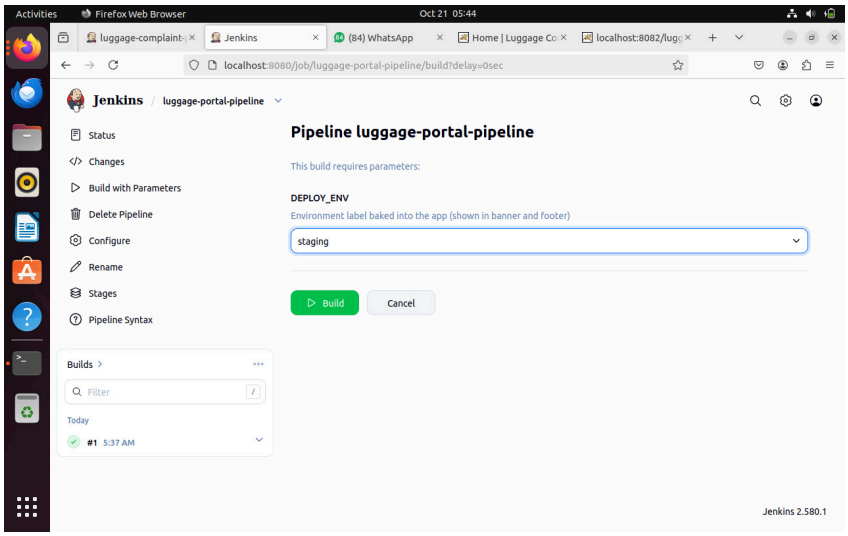


Figure: Build with Parameters: DEPLOY_ENV = staging

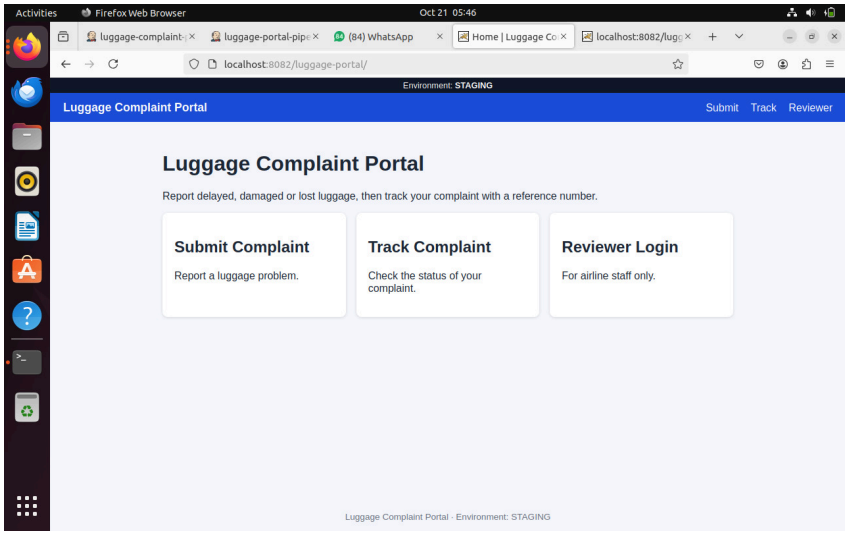


Figure: Deployed application showing Environment: STAGING

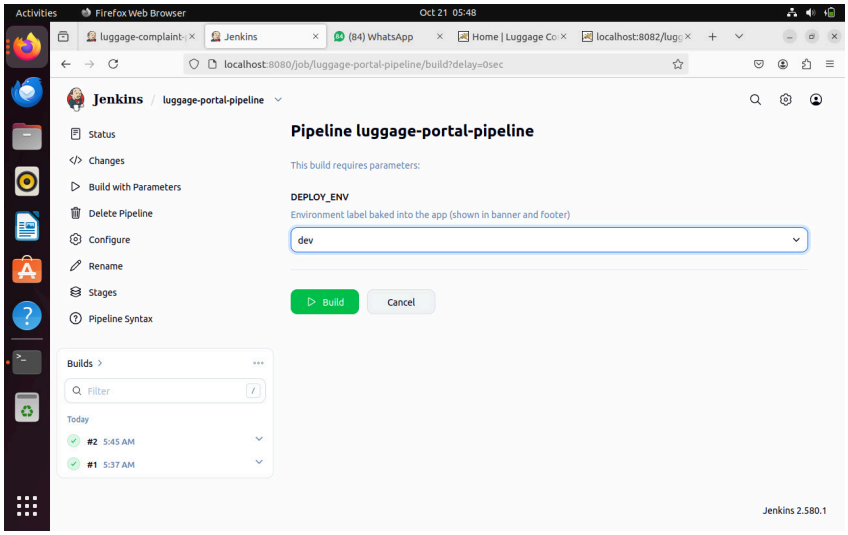
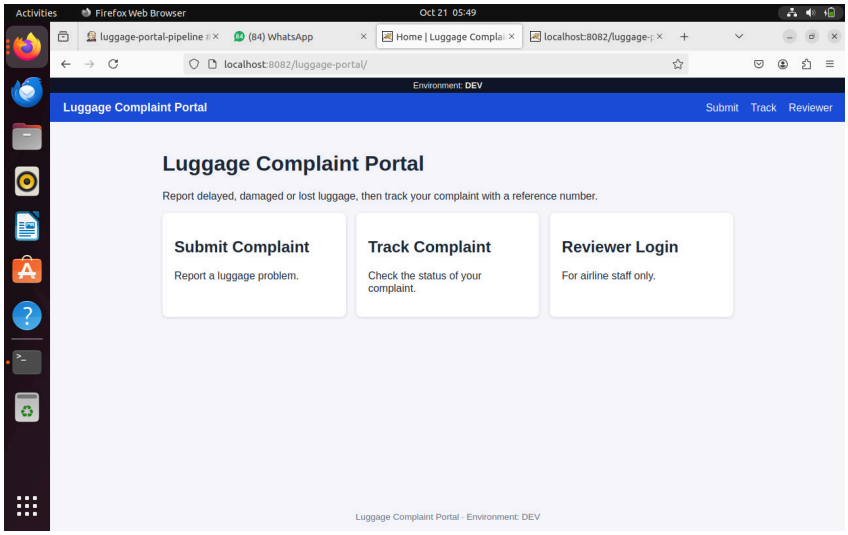


Figure: Build with Parameters: DEPLOY_ENV = dev

	 *Figure: Deployed application showing Environment: DEV*
Conclusion	A Jenkinsfile with checkout, build, package and deploy stages deployed the application to Tomcat, and the DEPLOY_ENV parameter changed the environment shown in the application.

VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	9
Experiment Title	Luggage Complaint Portal: Selenium Test Design and Local Execution
Objectives (Skill Set / Knowledge Tested / Imparted)	To identify critical user journeys, write Selenium WebDriver tests with assertions and test data, save screenshots on failure and run the suite locally through Maven.
Program	

1. Test plan

ID	Journey	Steps	Expected result
J1	Valid complaint shows a reference number	Open /complaints/new, fill all fields, submit	Reference in the format LCP-YYYY-NNN NNN
J2	Empty form shows validation errors	Open /complaints/new, submit empty	At least 5 validation errors
J3	Tracking shows the status	Submit a complaint, open /track, enter the reference	Status is SUBMITTED
J4	Reviewer approves a complaint	Submit, log in as reviewer, open complaint, choose APPROVE	Status becomes APPROVED

Test data: Asha Verma, asha.verma@example.com, 9876543210, flight AI202, PNR AB12CD, bag tag 1234567890, type DELAYED. Tool: Selenium WebDriver 4 with JUnit 5, class PortalJourneyTest.

2. Local execution through Maven

```
mvn -B test -Pselenium -Dtest=PortalJourneyTest
-Dbase.url=http://localhost:8082/luggage-portal
```

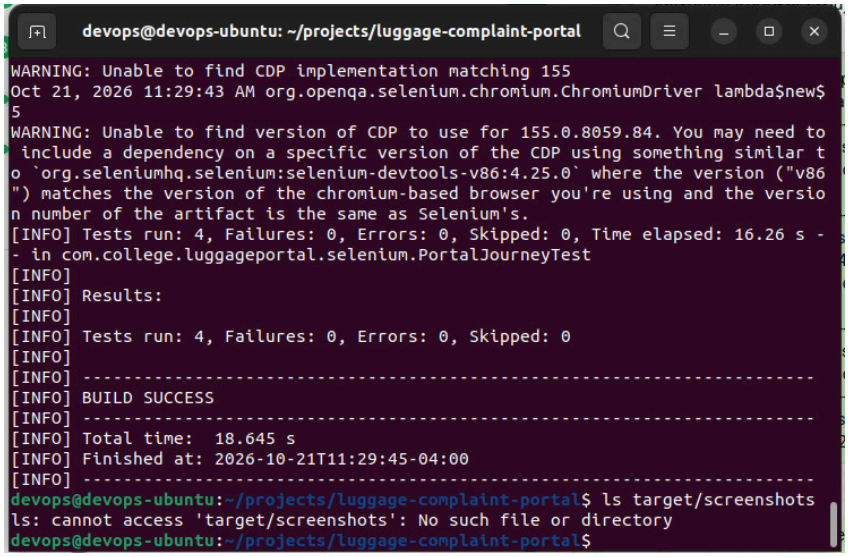


Figure: Tests run: 4, Failures: 0, Errors: 0 and BUILD SUCCESS

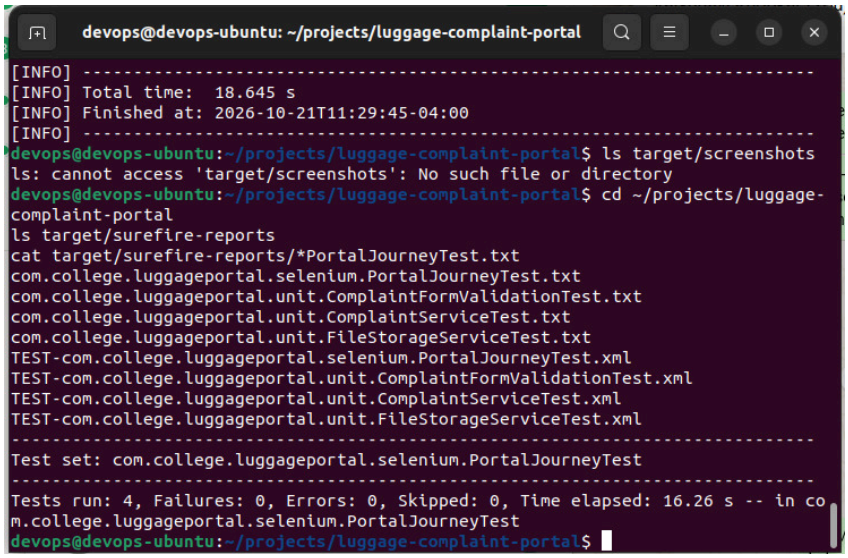


Figure: Surefire reports and the PortalJourneyTest report

3. Failure screenshot mechanism

ScreenshotOnFailure.java is a JUnit 5 extension that saves target/screenshots/<testName>.png when a test fails.

4. Deliberate defect to prove the mechanism

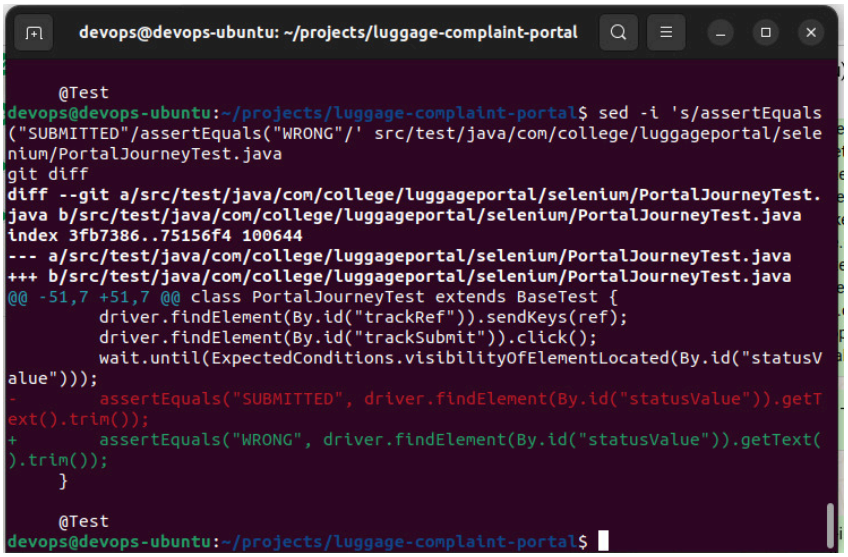


Figure: Expected status changed from SUBMITTED to WRONG in journey J3

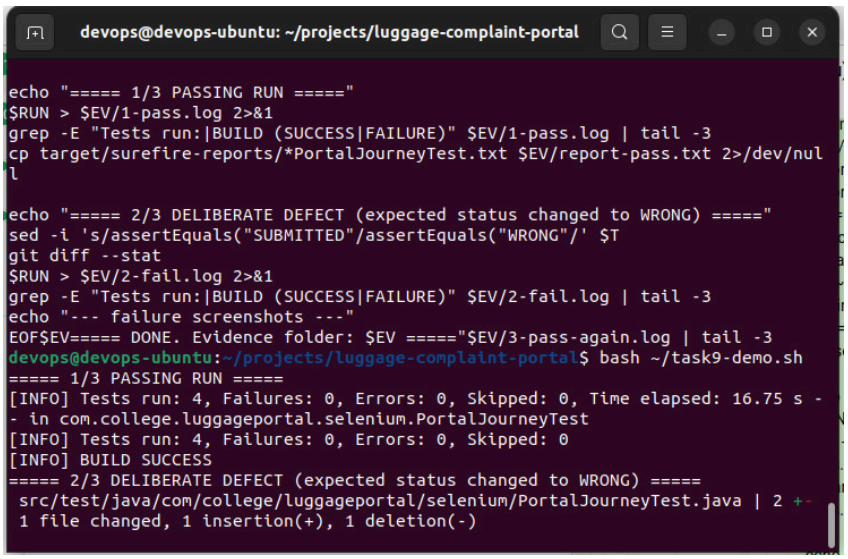


Figure: Demonstration script: passing run, then the deliberate defect

```
===== 1/3 PASSING RUN =====
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
===== 2/3 DELIBERATE DEFECT =====
Tests run: 4, Failures: 1, Errors: 0, Skipped: 0
BUILD FAILURE
--- failure screenshots ---
j3_trackingShowsStatus.png
===== 3/3 FIX AND RERUN =====
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

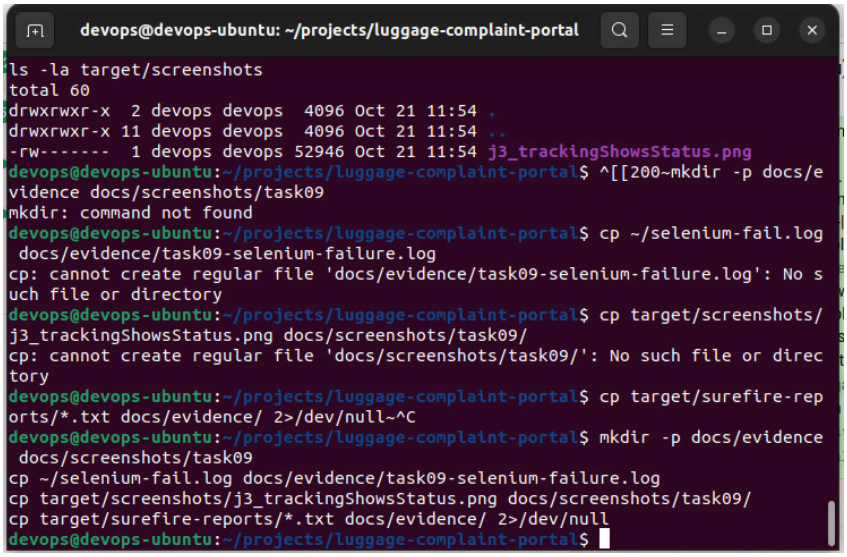


Figure: Saved failure screenshot in target/screenshots

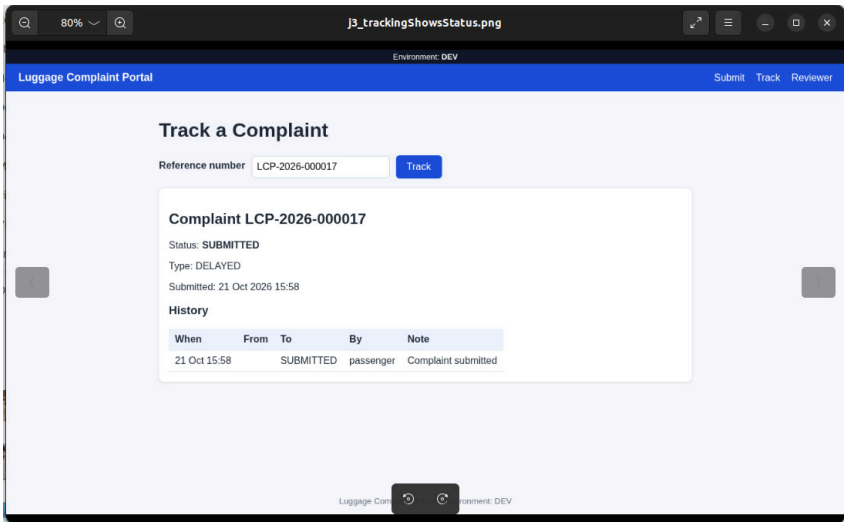


Figure: j3_trackingShowsStatus.png captured automatically at the failure

Conclusion	Four critical journeys were automated with Selenium and run through Maven. A screenshot is saved automatically when a test fails, and the suite passed again after the defect was removed.

Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	10
Experiment Title	Luggage Complaint Portal: Continuous Testing in Jenkins
Objectives (Skill Set / Knowledge Tested / Imparted)	To integrate the Selenium suite into Jenkins, publish test reports, stop deployment when tests fail, and correct a deliberately introduced defect and rerun the pipeline.
Program	**1. Pipeline with a Selenium quality gate** Stages: Checkout, Build, Unit Test, Package, Deploy to Test, Selenium Tests, Deploy Final. Selenium runs against a separate test deployment (/luggage-test); Deploy Final runs only when every test passed. **2. Successful run (build #4)**

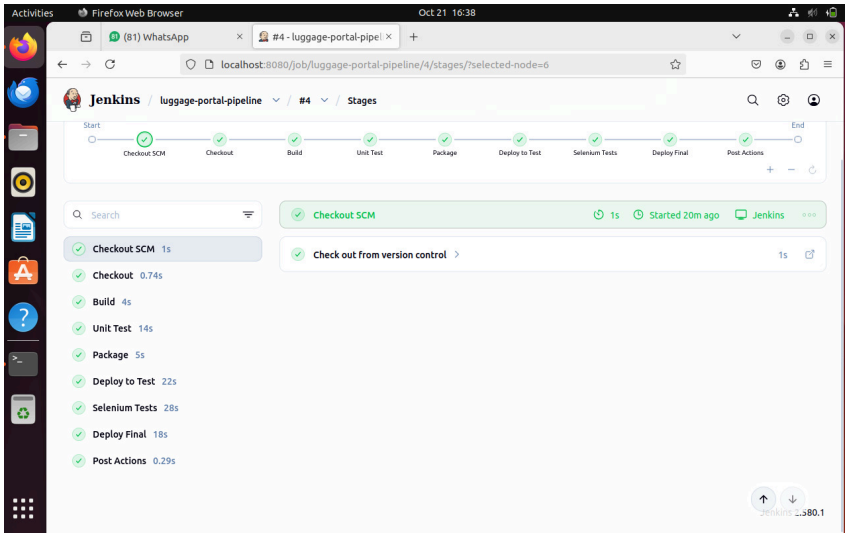


Figure: Build #4 stages: all stages green including Selenium Tests and Deploy Final

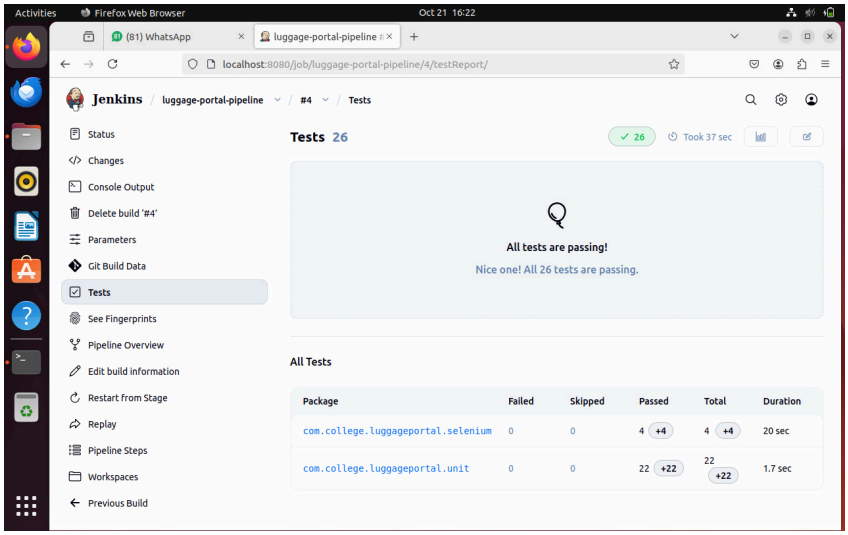


Figure: Jenkins test report: 26 tests passing (22 unit and 4 Selenium)

3. Defect introduced: pipeline fails (build #5)

The id of the submit button in the complaint form was changed from submit to submitX, so the Selenium journeys could no longer find the button.

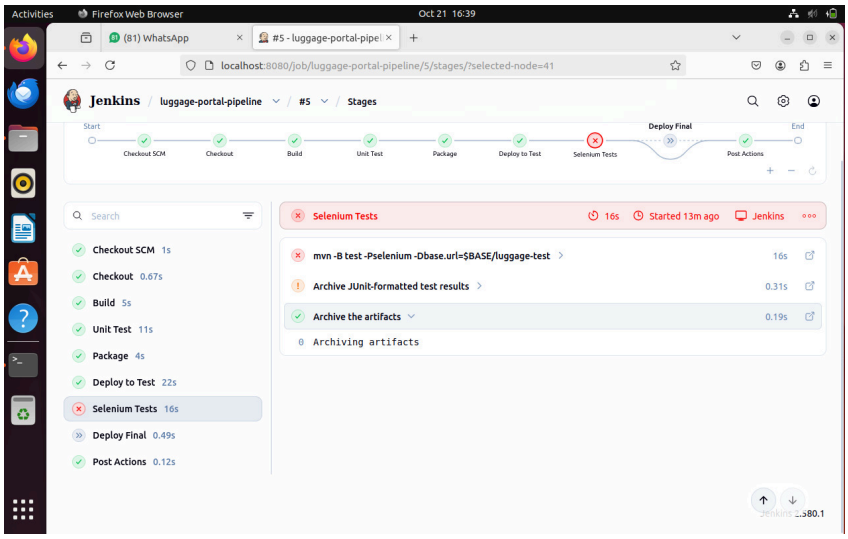


Figure: Build #5: Selenium Tests failed and Deploy Final was skipped

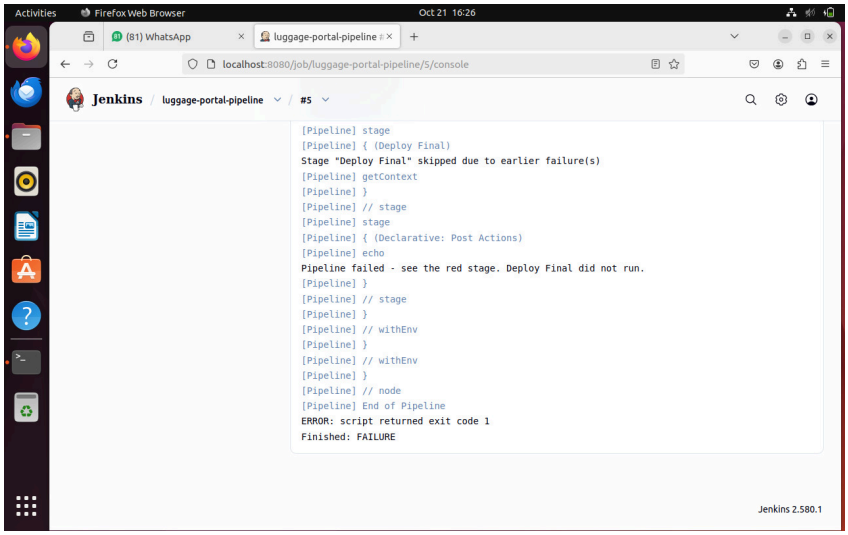


Figure: Console output: Deploy Final skipped due to earlier failures, Finished: FAILURE

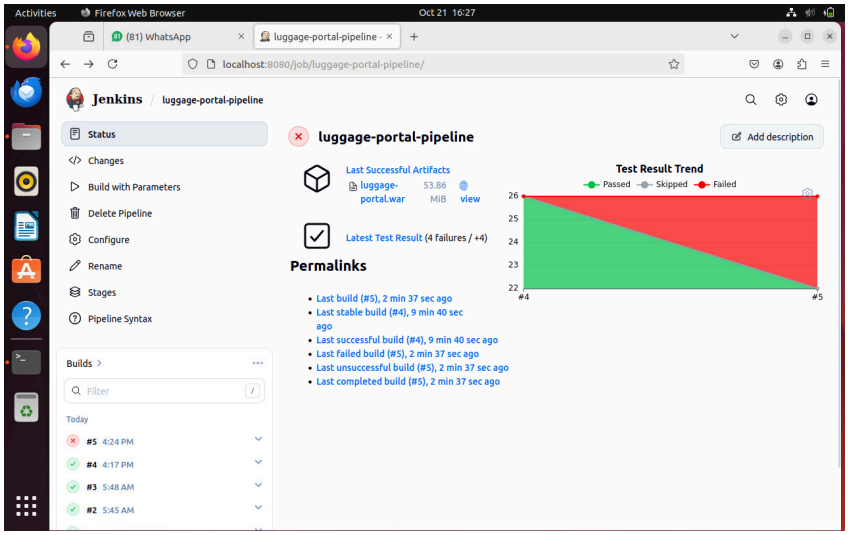


Figure: Job status after the failure with the test result trend

4. Defect correction commit

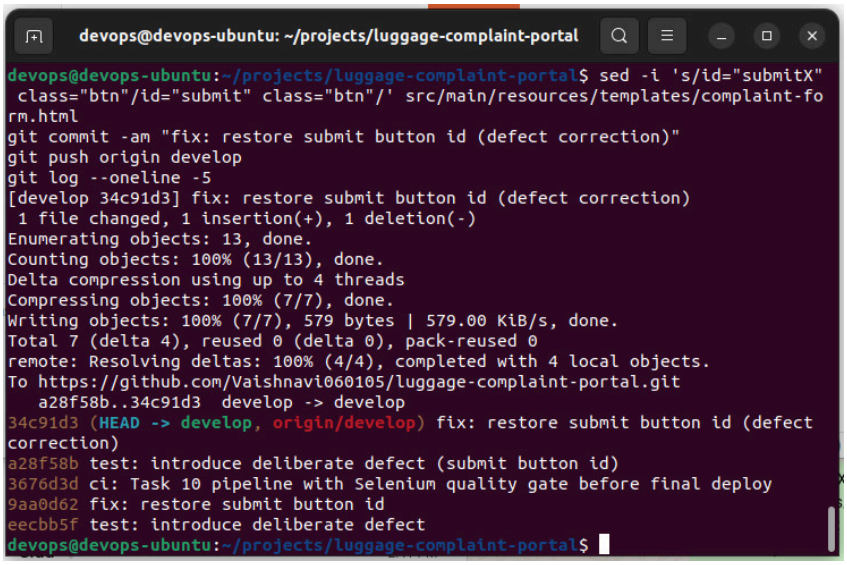


Figure: Commits: defect introduced (a28f58b) and fix: restore submit button id (34c91d3)

5. Successful rerun (build #6)

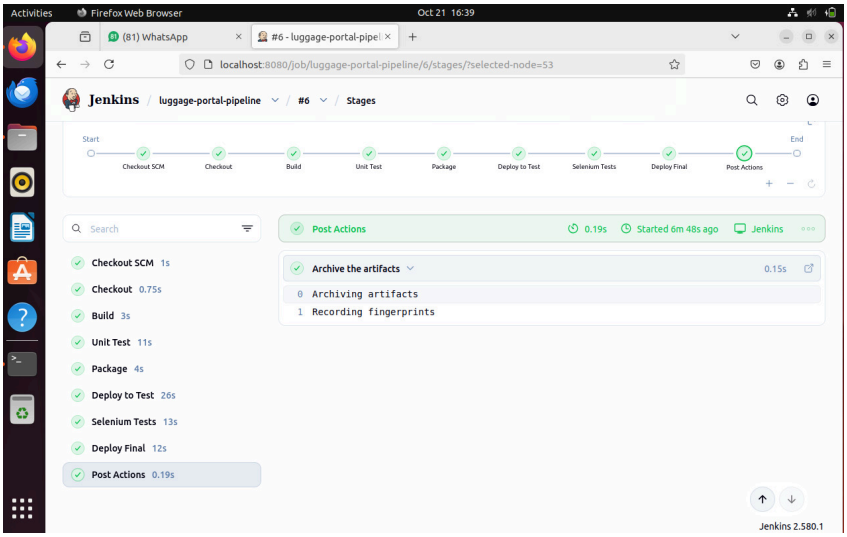


Figure: Build #6 stages: all green after the fix

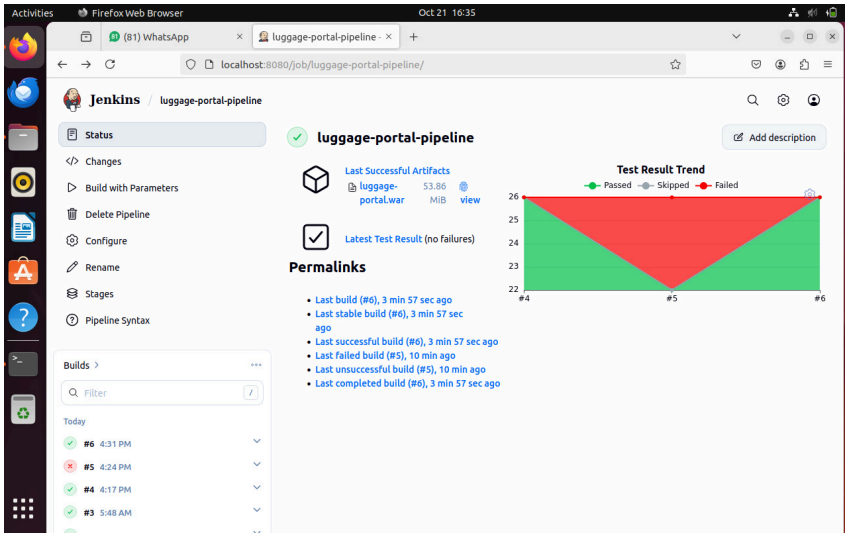


Figure: Job page showing the recovery: #4 passed, #5 failed, #6 passed

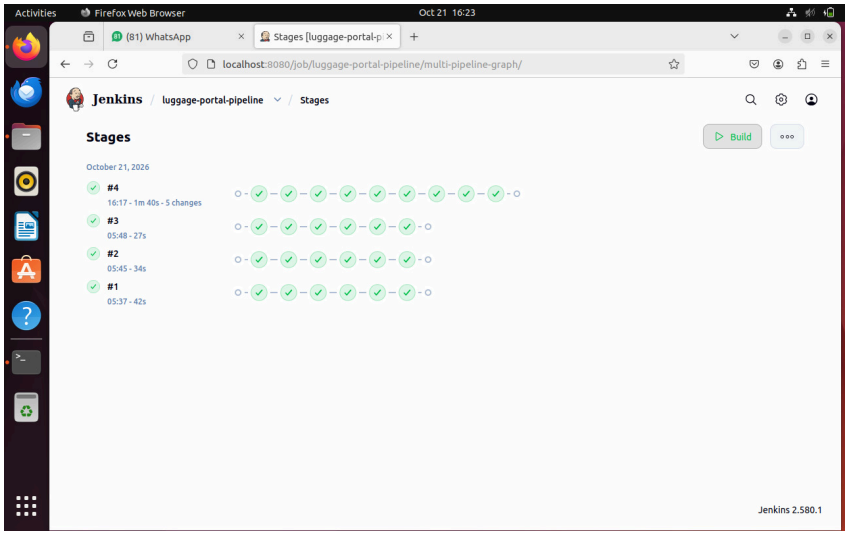


Figure: Stages overview of all builds

Conclusion	Selenium tests were added to the Jenkins pipeline as a quality gate. The deliberate defect stopped deployment, and after the correction commit the pipeline passed again.

Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	11
Experiment Title	Luggage Complaint Portal: Docker Image and Container Lifecycle
Objectives (Skill Set / Knowledge Tested / Imparted)	To create a Dockerfile, build and tag the image, run the container with port mapping, inspect logs, stop, restart and remove it, and document the full container lifecycle.
Program	

1. Dockerfile

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/luggage-portal.war app.war
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s --start-period=60s --retries=3 \
  CMD curl -f http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "app.war", "--server.port=8080"]
```

2. Build and tag the image

```
$ docker build -t luggage-portal:1.0 -t luggage-portal:latest .
Successfully built cdec67bfc9fe
Successfully tagged luggage-portal:1.0
Successfully tagged luggage-portal:latest

$ docker images luggage-portal
luggage-portal:1.0      cdec67bfc9fe   534MB (content 161MB)
luggage-portal:latest   cdec67bfc9fe   534MB (content 161MB)

$ docker image inspect luggage-portal:1.0 --format "ID: {{.Id}} Size:
{{.Size}}"
ID: sha256:cdec67bfc9fe...  Size: 160530746
```

3. Run the container with port mapping

```
$ docker run -d --name portal -p 8083:8080 luggage-portal:1.0
$ docker ps
d9e370e54043  luggage-portal:1.0  Up (health: starting)
0.0.0.0:8083->8080/tcp  portal
```

4. Logs, health and resource use

```
$ docker logs --tail 15 portal
... Tomcat started on port 8080 (http) with context path '/'
... Started LuggagePortalApplication in 4.714 seconds

$ curl -I http://localhost:8083/
HTTP/1.1 200

$ docker stats --no-stream portal
CPU 0.50%   MEM 303.8MiB / 3.824GiB   PIDS 39
```

5. Stop, start, restart and remove

```
$ docker stop portal
$ docker ps -a --filter name=portal
portal    Exited (143)
$ docker start portal
$ docker restart portal
$ docker ps
portal    Up 20 seconds (healthy)    0.0.0.0:8083->8080/tcp
$ docker stop portal
$ docker rm portal
$ docker ps -a --filter name=portal
(no containers)
```

6. Final running container

```
$ docker run -d --name portal -p 8083:8080 --restart unless-stopped
luggage-portal:1.0
$ docker ps
4e198e99d97b  luggage-portal:1.0  Up 30 seconds (healthy)
0.0.0.0:8083->8080/tcp  portal
```

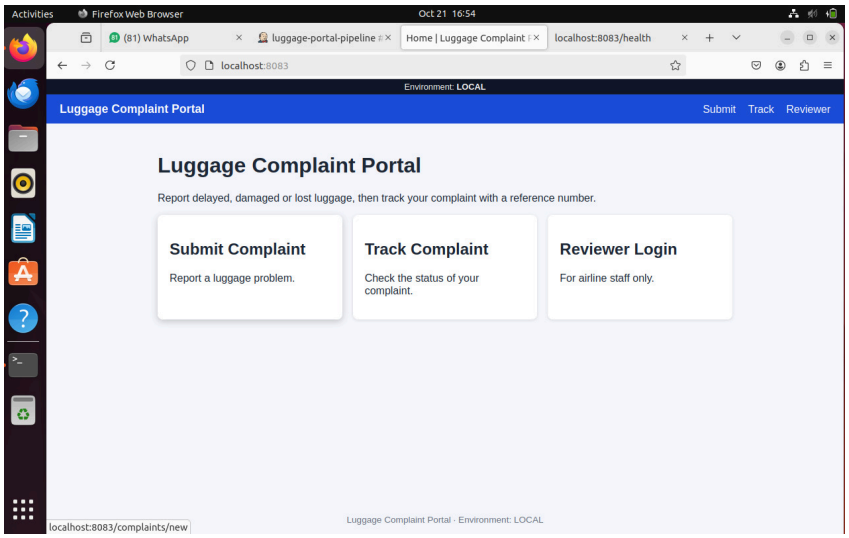


Figure: Application served from the container on port 8083

Conclusion	The application was packaged as a Docker image and the complete container lifecycle (build, run, port mapping, logs, stop, restart and remove) was carried out and logged.

Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	12
Experiment Title	Luggage Complaint Portal: Jenkins-Docker Continuous Deployment
Objectives (Skill Set / Knowledge Tested / Imparted)	To extend the Jenkins pipeline to build a versioned Docker image, publish it to a registry and deploy a fresh container automatically after the tests pass.

Program	**1. Local registry and Docker access for Jenkins**
	`docker run -d --name registry --restart unless-stopped -p 5000:5000 registry:2`
	`sudo usermod -aG docker jenkins`
	`sudo systemctl restart jenkins`
	2. Docker stages added to the Jenkinsfile
	`environment { IMAGE = 'localhost:5000/luggage-portal' TAG = "${env.BUILD_NUMBER}" // version tag = build number }`
	`stage('Docker Build') { steps { sh 'docker build -t $IMAGE:$TAG -t $IMAGE:latest .' } }`
	`stage('Docker Push') { steps { sh 'docker push $IMAGE:$TAG && docker push $IMAGE:latest' } }`
	`stage('Docker Deploy') { steps { sh 'docker rm -f portal || true docker run -d --name portal --restart unless-stopped -p 8083:8080 $IMAGE:$TAG' } }`

3. Versioned images in the registry

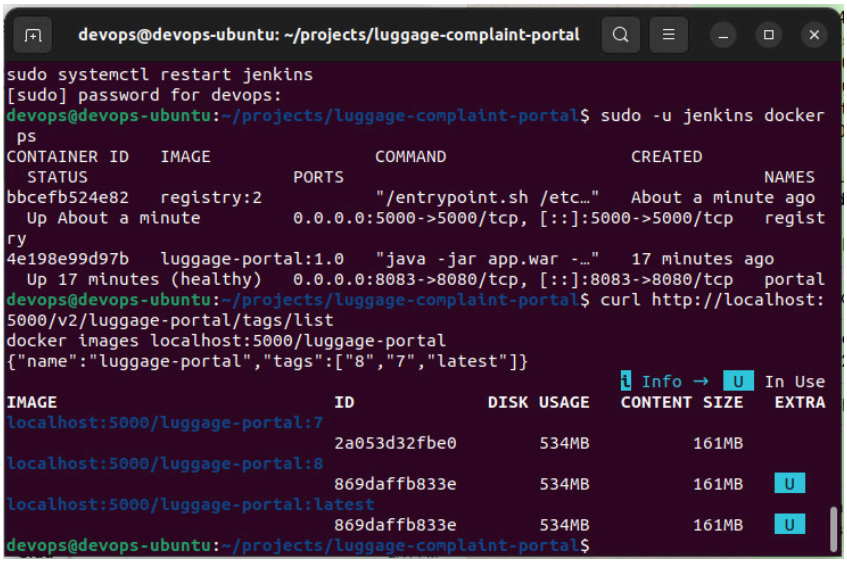


Figure: Registry tags ["8","7","latest"], docker images and the running portal container

4. End-to-end pipeline run

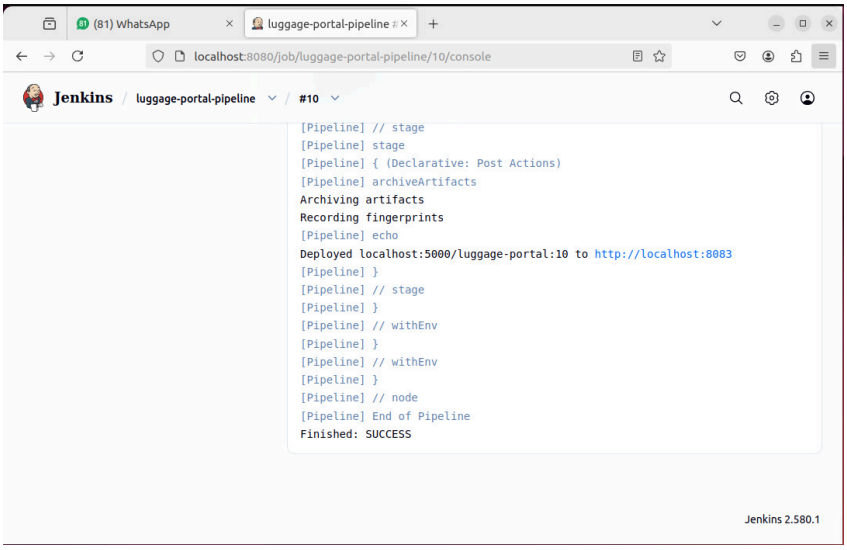
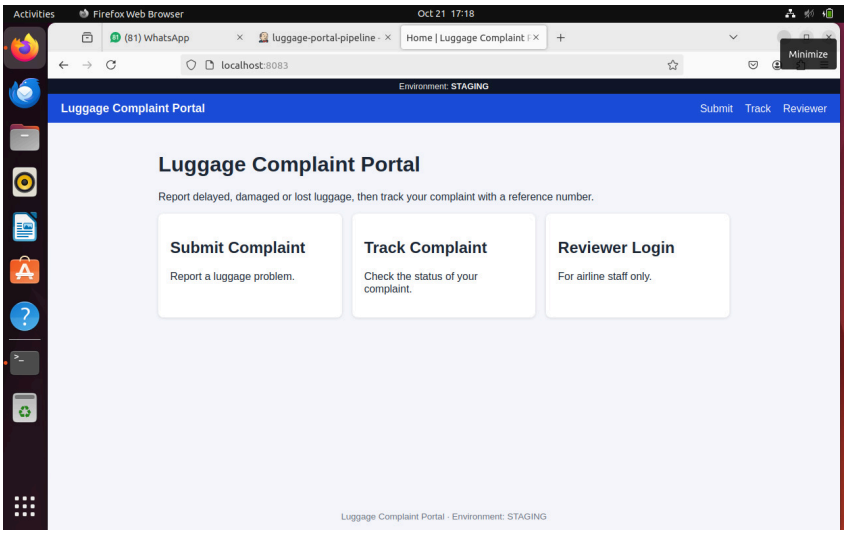


Figure: Build #10: Deployed localhost:5000/luggage-portal:10 to http://localhost:8083, Finished: SUCCESS

	5. Fresh container serving the new build  *Figure: Application on port 8083 showing Environment: STAGING*
Conclusion	The pipeline builds a versioned image, pushes it to a local registry and deploys a fresh container automatically after the tests pass, from commit to container.

VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	13
Experiment Title	Luggage Complaint Portal: Configuration Management Script
Objectives (Skill Set / Knowledge Tested / Imparted)	To identify the server prerequisites (packages, users, folders, files, ports, services) and write an Ansible inventory and playbook that configures the target node.
Program	

1. Configuration specification

Category	Requirement
Packages	openjdk-17-jre-headless, curl
User	portal (system user, shell /usr/sbin/nologin, no home)
Folders	/opt/luggage-portal, /opt/luggage-portal/releases, /opt/luggage-portal/uploads, /var/log/luggage-portal
Files	releases/portal-<version>.war, symlink current.war, /etc/systemd/system/luggage-portal.service
Port	8090 (8080 is used by Jenkins)
Service	luggage-portal enabled at boot and running; restarted when the release or unit changes
Health check	GET /actuator/health returns 200 and UP

2. Inventory

```
[portal]
localhost ansible_connection=local
```

3. Playbook tasks (ansible/site.yml)

- Install Java 17 and curl (apt).
- Create the service user portal.
- Create the application and log folders with the right owner and mode.
- Copy the WAR to releases/ (force=false keeps older releases, which makes rollback safe).
- Point current.war at the selected app_version.
- Install the systemd unit and notify the handler to restart the service.
- Enable and start the service, flush handlers, then run the health check with retries.

	4. First execution
	`ansible-playbook -i inventory.ini site.yml -K -e app_port=8090`
	The first run stopped at the first task because of an old GitHub CLI apt repository without a signing key:
	`TASK [Install Java 17 and curl]`
	`fatal: [localhost]: FAILED! ... GPG error: https://cli.github.com/packages stable InRelease:`
	`NO_PUBKEY 5612B36462313325 ... The repository is not signed.`
	Fix: the unused repository was removed (sudo rm /etc/apt/sources.list.d/github-cli.list), apt update was repeated and the playbook was run again.
	[Screenshot to be added: PLAY RECAP of the successful run (failed=0), the service status and the health check output]

Conclusion	The server prerequisites were specified and an Ansible playbook was written to configure the portal node. The first run exposed an apt repository key problem, which was identified and fixed.

VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	14
Experiment Title	Luggage Complaint Portal: Automated Provisioning and Reliability Validation
Objectives (Skill Set / Knowledge Tested / Imparted)	To provision a clean node with Ansible, deploy the application, rerun the automation to show idempotency, run a health check and test rollback and recovery to the previous stable release.

Program	**1. Provisioning design**
	• Clean node: service, unit file, folders and the portal user removed before the run.
	• Provisioning: ansible-playbook installs Java, the user, folders, the release, the systemd service and runs the health check.
	• Idempotency: a second run of the same playbook should report changed=0.
	• Rollback: running the playbook with the previous app_version repoints current.war and restarts the service.
	• Recovery: the unit uses Restart=always, so systemd restarts the service if it is killed.
	2. Commands used
	`AP="ansible-playbook -i ansible/inventory.ini ansible/site.yml -e app_port=8090"`
	`$AP -e app_version=1.0.0 # provision clean node`
	`$AP -e app_version=1.0.0 # idempotency: expect changed=0`
	`$AP -e app_version=1.1.0 # new release`
	`$AP -e app_version=1.0.0 # rollback to previous stable release`
	`sudo systemctl kill -s KILL luggage-portal # recovery test`
	`curl http://localhost:8090/actuator/health`
	3. Evidence
	[Screenshot to be added: Provisioning run on the clean node (PLAY RECAP)]
	[Screenshot to be added: Second run showing changed=0 (idempotency)]
	[Screenshot to be added: Health check output {"status":"UP"}]
	[Screenshot to be added: Rollback: actuator/info before and after, and the releases folder listing]
	[Screenshot to be added: Recovery: service active again after being killed]

Conclusion	The provisioning playbook installs the application as a versioned, health-checked systemd service. Rerunning it changes nothing, and a rollback is done by running it with the previous version.

VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	Department of Computer Engineering A.Y-2026-27

Semester	B.E. Semester VII – CMPN
Subject	DevOps
Subject Professor In-charge	Prof. Snehal Andhare
Assisting Teachers	Prof. Snehal Andhare
Laboratory	

Student Name	Vaishnavi Khopkar	
Roll Number	23102A0074	
Grade and Subject Teacher's Signature		

Experiment Number	15
Experiment Title	Luggage Complaint Portal: Final End-to-End Release, Documentation and Viva
Objectives (Skill Set / Knowledge Tested / Imparted)	To run the complete DevOps workflow from Git commit to Jenkins build, Selenium quality gate, Docker deployment and Ansible provisioning, and to complete the documentation.
Program	**1. End-to-end workflow**

Stage	Tool	Result
Commit and push	Git, GitHub	Code on the develop branch, release tagged v1.0.0
Build and unit tests	Jenkins, Maven, JUnit	22 unit tests passing
Quality gate	Selenium	4 journeys passing; failed tests stop deployment
Deploy	Tomcat	Application on port 8082
Package and publish	Docker, local registry	Versioned images (tags 7, 8, 10, latest)
Run	Docker container	Application on port 8083
Provision	Ansible	Configured node with health check and rollback

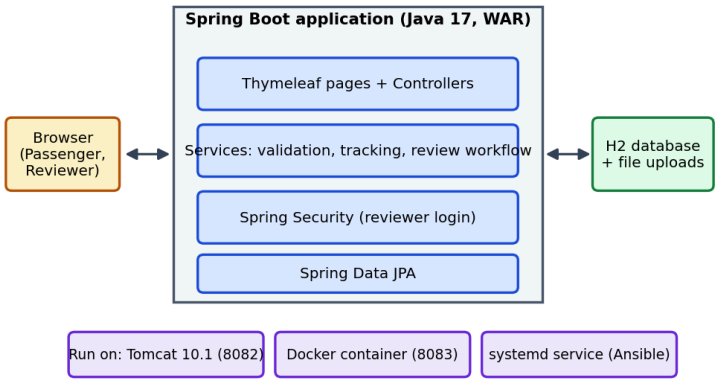


Figure: Architecture of the portal and its deployment targets

2. Final pipeline run

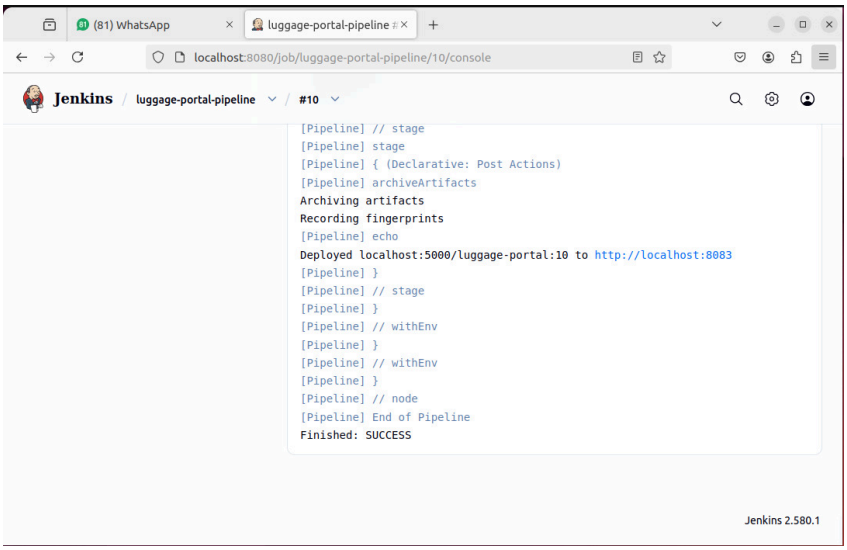


Figure: Build #10: commit to container, Finished: SUCCESS

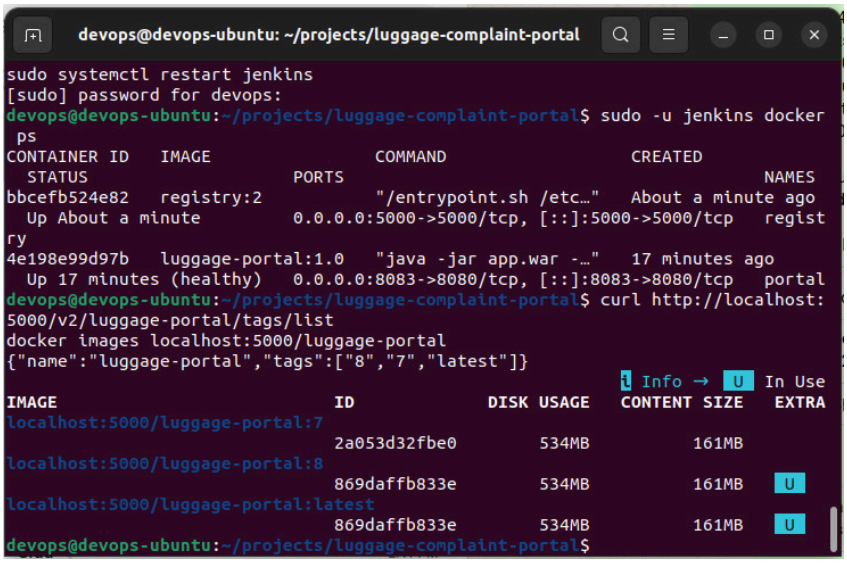


Figure: Versioned images in the registry and the running container

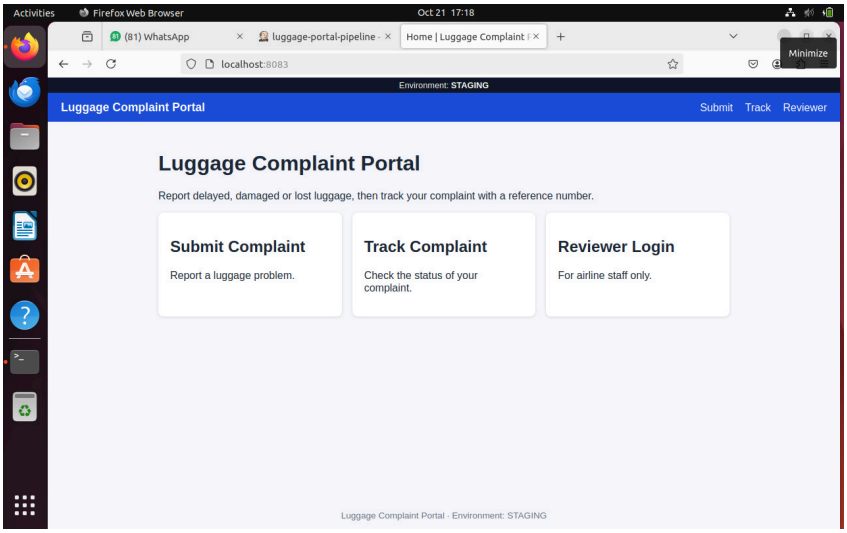


Figure: Final application on port 8083

3. Troubleshooting guide

Problem	Cause	Fix
release version 17 not supported	Default Java was a Java 21 runtime without compiler	Set JAVA_HOME to Java 17
Could not resolve host	VM DNS dropped	Set DNS 8.8.8.8 and 1.1.1.1
Jenkins archive found nothing	Project builds a WAR, not a JAR	Archive target/*.war
Permission denied in /opt/tomcat	Folder owned by the tomcat user	Add jenkins to the tomcat group, chmod 755
Docker permission denied	User not in the docker group	usermod -aG docker, restart Jenkins

Port already in use	Several services on one VM	Fixed port plan: 8080, 8081, 8082, 8083, 8090, 5000
apt NO_PUBKEY (Ansible)	Leftover GitHub CLI repository	Remove the repository file

4. Limitations

- H2 embedded database: data is not suitable for production.
- Everything runs on a single VM.
- Local registry without TLS or authentication.
- Demo reviewer account and no real airline integration.
- Builds are started by polling or manually, not by webhooks.

5. Future enhancement plan

- PostgreSQL with persistent storage and backups.
- Email and SMS notifications to passengers.
- GitHub webhooks and Jenkins agents on separate nodes.
- Kubernetes deployment, HTTPS and secrets management.
- Monitoring and alerting (Prometheus and Grafana).

[Screenshot to be added: Ansible provisioning run, live demonstration video and viva presentation]

Conclusion	The complete workflow ran from a commit through build, test, a versioned Docker image and a fresh container. Troubleshooting notes, limitations and a future enhancement plan are documented.